

ATELIER C++



```
#include <iostream>

int main() {
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```



Plan du cours

- Introduction générale
 - Programmation Procédurale
 - Programmation Orienté Objet POO
- Langage C++ : Notion de base
- Éléments de langage

PROGRAMMATION PROCEDURALE (1)

- La **programmation procédurale** est un **paradigme de programmation** basé sur l'organisation du code en **procédures ou fonctions**.
- Une **procédure** (ou fonction) est un ensemble d'instructions qui effectue une tâche spécifique.
- Les programmes procéduraux sont généralement **linéaires** et **structurés**.

Identifier le
problème

Décomposer le
problème en
sous problèmes

Définir les
fonctions
correspondantes

Ecrire le
programme
principal en
pseudo code

Implémenter
Exécuter
Débugger

PROGRAMMATION PROCEDURALE (2)

- **Caractéristiques principales**

- 1. Organisation en fonctions :**

- Chaque tâche est isolée dans une fonction.
- Exemple : une fonction pour additionner deux nombres, une autre pour afficher un message.

- 2. Séquence d'instructions :**

- Le programme suit un flux séquentiel : exécution des instructions **ligne par ligne**.

- 3. Données globales ou locales :**

- Les **variables** peuvent être globales (accessibles partout) ou locales (accessibles seulement dans une fonction).

- 4. Contrôle de flux :**

- Utilisation de **conditions** (if, switch) et **boucles** (for, while) pour contrôler l'exécution du programme.

- 5. Réutilisation limitée :**

- Les fonctions permettent de réutiliser du code, mais la gestion de grandes quantités de données devient complexe.

PROGRAMMATION PROCEDURALE (3)

Avantages



- ✓ Facile à apprendre
- ✓ Réutilisation et maintenabilité claire
- ✓ Approche séquentielle claire
- ✓ Organisation et clarté du code

Limites



- × Difficulté à gérer des programmes complexes ou très grands
- × Difficile de représenter des objets du monde réel (comme un employé, une voiture, etc.)
- × Moins flexible pour la **réutilisation de code** comparé à la programmation orientée objet

Transition vers la Programmation Orientée Objet (POO)

Pourquoi passer à la POO ?

- La **programmation procédurale** fonctionne bien pour des programmes simples.
- Mais quand un programme devient **grand et complexe**, il devient difficile de gérer :
 - Les **données** dispersées
 - Les **fonctions dépendantes**
 - La **réutilisation du code**
- **La solution** : regrouper les données et les fonctions qui les manipulent dans une même entité, appelée objet.

PROGRAMMATION ORIENTE OBJET POO (1)

- La **Programmation Orientée Objet (POO)** est un **paradigme de programmation** basé sur l'organisation du code autour des objets, qui représentent des entités du monde réel.
- **Objet** : instance d'une classe, contenant **données (attributs)** et **comportements (méthodes)**.
- **Classe** : modèle ou plan de construction pour créer des objets.
- **Exemple conceptuel** :

Objet réel	Attributs(données)	Méthodes
Voiture	couleur, marque, vitesse	Démarrer(), freiner(), klaxonner()
Employé	nom, âge , salaire	Travailler(), sePrésenter()

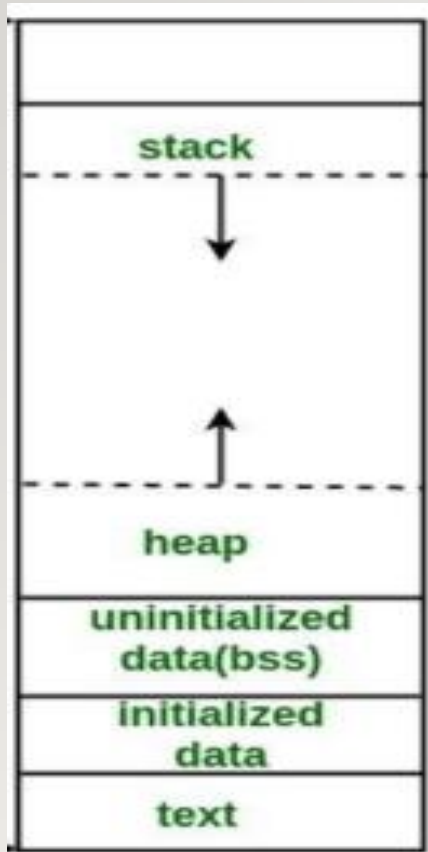
PROGRAMMATION ORIENTE OBJET POO (2)

- Les points forts du POO :
 - **Modularité** : les objets forment des modules compacts regroupant des données et un ensemble d'opérations.
 - **Abstraction** : Les entités objets de la POO sont proches de celles du monde réel. Les concepts utilisés sont donc proches des abstractions familières que nous exploitons.
 - **Productivité et réutilisabilité** : Plus l'application est complexe et plus l'approche POO est intéressante en termes de productivité.
 - **Sûreté** : L'encapsulation et le typage des classes offrent une certaine robustesse aux applications.

Qu'est-ce que le C++ ?

- Langage compilé, dérivé du C mais avec la programmation orientée objet.
- Un programme écrit en langage C++ ressemble beaucoup à un programme écrit en langage C, à la différence près qu'il contient essentiellement des classes.
- Utilisé en : systèmes embarqués, jeux vidéo, applications hautes performances, etc.
- Un programme C++ est constitué au minimum d'une fonction main().
- Le langage C++ possède assez peu d'instructions, il fait par contre appel à des bibliothèques
- **Exemples:**
 - math.h : bibliothèque de fonctions mathématiques
 - iostream.h : bibliothèque d'entrées/sorties standard
 - complex.h : bibliothèque contenant la classe des nombres complexes.

FORME DE LA MÉMOIRE EN PROGRAMME C++



- **Code (ou Text Segment)** : Contient le **code compilé** du programme (les instructions machine). C'est une zone **en lecture seule** → on ne peut pas la modifier pendant l'exécution.

Exemple : la fonction `main()` ou `cout << "Hello";`.

- **Données statiques (Data Segment)** : Stocke les **variables globales, statiques** et les **constantes**. Il existe sous-parties :
 - **Initialized Data** → variables globales ou statiques avec une valeur initiale.
 - **Uninitialized Data (BSS)** → variables globales ou statiques **non initialisées** (remplies par défaut avec 0).
- **Pile (Stack)** : allocation temporaire des objets (existent seulement durant l'exécution d'une fonction).
- **Tas (Heap)** : allocation dynamique des objets (le programmeur alloue et désalloue la mémoire, opérateur `delete` et `free`)

STRUCTURE D'UN PROGRAMME EN C++

```
main.cpp x
1  #include <iostream>
2  #include "fcts.h"
3  using std::cout;
4  using std::cin;
5  using std::endl;
6
7  int main()
8  {
9      int a,b;
10     cout << "Entrer un entier" << endl;
11     cin >> a;
12     cout << "Entrer un entier" << endl;
13     cin >> b;
14
15     cout << "La somme est " << somme(a,b) << endl;
16     return 0;
17 }
```

Directive préprocesseur. Utiliser une bibliothèque **standard** `iostream`

Utilisation d'une bibliothèque personnelle

Utilisation des objets prédéfinis dans le nom de domaine **std**

Fonction principale du programme

cout : la sortie standard du programme (console)

cin : l'entrée standard des données saisies au clavier

endl : fin de la ligne

<< : opérateur de sortie du flux de données

>> : opérateur d'entrée du flux de données

CONCEPT DE BASE : INITILISATION

3 manières d'initialiser une variable :

(*expression-list*) (1)

= *expression* (2)

{ *initializer-list* } (3)

```
int b = 1;  
int c(2);  
int d{}; // d vaut 0
```

```
std::cout << "b= " << b << std::endl;  
std::cout << "c= " << c << std::endl;  
std::cout << "d= " << d << std::endl;
```

Console c

```
b= 1  
c= 2  
d= 0
```


ALLOCATION STATIQUE – ALLOCATION DYNAMIQUE

Allocation statique

```
int x =0 ;  
int y(5);
```

Les variables sont allouées de manière permanente.

L'allocation est faite avant l'exécution du programme.

Pas de possibilité de réutilisation de la mémoire.

Allocation dynamique

```
int *pt = new int(10);
```

Les variables ne sont allouées que si votre unité de programme est active.

L'allocation est faite pendant l'exécution du programme.

La mémoire est réutilisable et peut être libérée lorsqu'elle n'est pas nécessaire (opérateur delete).

En c/c++, la gestion de la mémoire est à la charge du programmeur. Il doit libérer de la mémoire à chaque fois qu'elle n'est plus pointée.

CONCEPT DE BASE : LES BOOLEENS

```
#include <iostream>

using namespace std;

int main()
{
    bool v_bool1 = true;
    bool v_bool2(false);

    cout << v_bool1 << endl;
    cout << v_bool2 << endl;
    cout << boolalpha << v_bool1 << endl;
    cout << boolalpha << v_bool2 << endl;
    return 0;
}
```

D:\Git\dev-isimg\AtelierCPP\fiche3\bin\Debug\fiche3.exe

1
0
true
false

Déclaration et initialisation

Une variable de type **bool** ne peut avoir que deux états
true (1) ou **false** (0)

CONCEPT DE BASE : LES STRINGS (1)

```
#include <iostream>

using namespace std;

int main()
{
    string lang("C plus plus");
    string niv = "CPI2";
    cout << niv << " " << lang << endl;
    // ajouter à la fin
    cout << niv.append(lang) << endl;
    // on peut utiliser aussi size() pour avoir la longueur
    cout << "La chaine contient " << niv.length() << " caracteres" << endl;
    return 0;
}
```

D:\Git\dev-isimg\AtelierCPP\fiche3\bin\Debug\fiche3.exe
CPI2 C plus plus
CPI2C plus plus
La chaine contient 15 caracteres

Opérateurs et fonctions :

+ : concaténation

length() / size() : longueur

== / != : comparaison

compare(str) : comparer

at(index) : lire un caractère

insert(index,str) : insérer

Plus de fonctions via ce lien:

<https://devdocs.io/cpp/>

Remarque: on peut parcourir un objet de type string avec un indice comme un tableau de caractères.

CONCEPT DE BASE : LES STRINGS (2)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string prenom = "Amani";
    string nom = "Fallah";

    string identite = prenom + " " + nom; // concaténation avec +

    cout << identite << endl; // affiche : Amani Fallah
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string phrase = "Bonjour";
    phrase.append(" tout le monde!");

    cout << phrase << endl; // affiche : Bonjour tout le monde!
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string a = "chat";
    string b = "chien";

    cout << a.compare(b) << endl; // renvoie > 0 car "chat" > "chien"
    cout << b.compare(a) << endl; // renvoie < 0 car "chien" < "chat"
    cout << a.compare("chat") << endl; // renvoie 0 car elles sont égales
}
```

CONCEPT DE BASE : LES STRINGS (3)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string mot = "programmation";

    cout << mot.at(0) << endl; // p (premier caractère)
    cout << mot.at(3) << endl; // g (4ème caractère)
    cout << mot.at(mot.size()-1) << endl; // n (dernier caractère)
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string mot = "programmation";

    mot.insert(6, "C++ ");
    cout << mot << endl; // affiche "prograC++ mmation"
}
```

CONCEPT DE BASE : LES STRINGS (4)

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string lang;
```

```
    cout << "C'est quoi votre langage prefere?" << endl;
```

```
    cin >> lang;
```

```
    cout << "Vous aimez programmer avec " << lang << endl;
```

```
    return 0;
```

```
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\Debug\fiche3.exe

C'est quoi votre langage prefere?
C plus plus
Vous aimez programmer avec C

cin >> : arrête la lecture au premier caractère espace

getline(cin, str): arrête la lecture au premier retour à la ligne

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string lang;
```

```
    cout << "C'est quoi votre langage prefere?" << endl;
```

```
    getline(cin, lang);
```

```
    cout << "Vous aimez programmer avec " << lang << endl;
```

```
    return 0;
```

```
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\Debug\fiche3.exe

C'est quoi votre langage prefere?
C plus plus
Vous aimez programmer avec C plus plus

LES CONSTANTES

En C++ on distingue deux grands types de **constantes** selon le moment où leur valeur est fixée

Compile-time constant (constante connue à la compilation)

```
#include <iostream>
using namespace std;

int main() {
    const int TAILLE = 10;      // constante de compilation
    constexpr double PI = 3.1415; // constante de compilation aussi

    int tab[TAILLE]; // possible car TAILLE est connu à la compilation

    cout << "PI = " << PI << endl;
}
```

Runtime constant (constante connue à l'exécution)

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Entrez une valeur : ";
    cin >> n;

    const int valeur = n; // runtime constant

    cout << "Vous avez saisi : " << valeur << endl;
}
```

Remarque : on peut utiliser constexpr seulement avec compile-time constant
const = la valeur ne change pas une fois affectée, mais elle peut être connue seulement à l'exécution.
constexpr = la valeur doit être connue à la compilation

LES TABLEAUX : RAW ARRAYS

```
constexpr int SIZE = 5;
int t[SIZE];

// initialiser t
for (int i = 0; i < SIZE; i++)
    t[i] = i;

// afficher t
for (int i = 0; i < SIZE; i++)
    std::cout << t[i] << " ";
```

Stack Memory Allocation

```
constexpr int SIZE = 5;
int* tab = new int[SIZE];

// initialiser tab
for (int i = 0; i < SIZE; i++)
    tab[i] = i;

// afficher tab
for (int i = 0; i < SIZE; i++)
    std::cout << tab[i] << " ";

delete[] tab;
```

Heap Memory Allocation

LES TABLEAUX : STATIC ARRAYS

```
#include <iostream>
#include <array>
#include <algorithm>

int main()
{
    std::array<int, 5> tab{33,5,8,-1, 88};

    // trier tab avec la fonction prédéfinie sort
    std::sort(tab.begin(), tab.end());

    // afficher tab
    for (int i = 0; i < tab.size(); i++)
        std::cout << tab[i] << " ";

    return 0;
}
```

Autre façon de parcourir

```
for (const auto& ele : tab)
    std::cout << ele << " ";
```

Stack Memory Allocation

LES CONTENEURS

En C++, un **conteneur** est une classe de la bibliothèque standard (STL – *Standard Template Library*) qui sert à **stocker et organiser des collections d'objets**.

➤ Exemples de conteneurs :

`std::vector` (tableau dynamique)

`std::list` (liste chaînée)

`std::map` (dictionnaire clé/valeur)

`std::set` (ensemble sans doublons)

LES CONTENEURS - VECTOR

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;           // vecteur vide d'entiers
    v.push_back(10);        // ajoute 10 à la fin
    v.push_back(20);
    v.push_back(30);

    cout << "Taille = " << v.size() << endl; // 3
    cout << "v[1] = " << v[1] << endl;      // 20

    v.pop_back();           // enlève le dernier élément (30)
}
```

- ✓ Le vector est le conteneur le plus proche d'un **raw array dynamique**, mais avec beaucoup plus de fonctionnalités.
- ✓ Un **vector** peut **grandir et rétrécir automatiquement** avec :
 - push_back() → ajoute à la fin
 - pop_back() → enlève le dernier élément
 - resize() → change la taille
- ✓ Quelques fonctions utiles de vector :
 - v.size() → nombre d'éléments
 - v.empty() → vrai si le vecteur est vide
 - v.clear() → supprime tous les éléments
 - v.front() → premier élément
 - v.back() → dernier élément
 - v.at(i) → élément avec vérification de la borne (plus sûr que v[i])
 - v.insert(pos, val) → insère un élément à une position donnée
 - v.erase(pos) → supprime un élément à une position

LES CONTENEURS - VECTOR

```
#include <iostream>
#include<vector>
using namespace std;
int main()
{
    vector<int> vl;
    // remplissage de vl
    for (int i = 1; i <= 5; i++)
        vl.push_back(i);
    // affichage avec un itérateur
    cout << "affichage dans l'ordre" << endl;
    for (auto it = vl.begin(); it != vl.end(); ++it)
        cout << *it << " ";
    cout << endl;
    cout << "affichage dans l'ordre inverse" << endl;
    for (auto it = vl.rbegin(); it != vl.rend(); ++it)
        cout << *it << " ";
    return 0;
}
```

D:\Git\dev-ising\AtelierCPP\fiche3\bin\De

affichage dans l'ordre

1 2 3 4 5

affichage dans l'ordre inverse

5 4 3 2 1

Inclure la bibliothèque vector.

Voir :

<https://cplusplus.com/reference/vector/vector/>

Création statique d'un tableau d'entiers

Insérer des éléments dans le tableau

Début du tableau partant de la gauche

Fin du tableau partant de la gauche

Début et fin du tableau partant de la droite.

LES CONTENEURS – MAP / UNORDRED MAP

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> ages;
    ages["Ali"] = 25;
    ages["Sara"] = 30;
    ages["Omar"] = 22;

    cout << "Age de Sara = " << ages["Sara"] << endl; // 30
}
```

- C'est comme un dictionnaire :
- Chaque **clé** est associée à une **valeur**.
- Les clés sont **uniques** et toujours **triées** automatiquement.

LES CONTENEURS – EXERCICES

Etant donné un tableau d'entiers « tab » contenant les N (saisie au clavier) premiers entiers partant de 0. on veut l'éclater en deux autres tableaux de manière à avoir un tableau d'entiers premiers et un tableau pour les entiers non premiers.

- Ecrire la fonction « estPremier » qui retourne un booléen indiquant si l'entier est premier ou non.
- Ecrire une fonction « remplir » qui lit la valeur maximale N et remplit le tableau passé en paramètre.
- Ecrire une fonction « afficher » qui affiche les éléments d'un tableau.
- Ecrire la fonction principale main qui crée et remplit le tableau « tab » et l'éclater en deux autres tableaux telle que demander.

LES CONTENEURS – EXERCICES

```
#include <iostream>
#include <cmath>
#include <vector>
bool estPremier(int& n) {
    bool prem = true;
    for (int i = 2; i <= sqrt(n); i++)
        if (n % i == 0)
            prem = false;
    return prem;
}
```

```
void remplir(std::vector<int>& v) {
    int n;
    std::cout << "Donner la limite des entiers:\n";
    std::cin >> n;
    for (int i = 0; i < n; i++)
        v.push_back(i);
}
```

```
void afficher(std::vector<int>& vec) {
    for (int& el : vec)
        std::cout << "[" << el << "]";
    std::cout << std::endl;
}
```


LES CONTENEURS – EXERCICES

```
int main()
{
    std::vector<int> tab{};
    remplir(tab);
    std::vector<int> vecPremier{}, vecNonPremier{};
    for (int& ele : tab)
        if (estPremier(ele) == true)
            vecPremier.push_back(ele);
        else
            vecNonPremier.push_back(ele);
    std::cout << "Les nombres premiers:\n";
    afficher(vecPremier);
    std::cout << "Les nombres non premiers:\n";
    afficher(vecNonPremier);
    return 0;
}
```


UNE VARIABLE PLUSIEURS TYPE

Depuis c++17, il est possible d'avoir une seule variable capable d'avoir plusieurs types.

```
#include <iostream>
#include <variant>

int main()
{
    std::variant<std::string, int, float> data;

    data = std::string("hello");
    std::cout << std::get<std::string>(data) << std::endl;

    data = 120;
    std::cout << std::get<int>(data) << std::endl;

    data = 1.03f;
    std::cout << std::get<float>(data) << std::endl;
    return 0;
}
```

std::variant<>

Accès aux membres du variant

UNE VARIABLE PLUSIEURS TYPE - EXERCICE

Soit **f** une fonction définie par $\sqrt{(x - 1) * (2 - x)}$

La fonction retourne une valeur réelle lorsqu'elle est définie, sinon elle retourne un message d'erreur « la fonction n'est pas définie !! ».

Ecrire une fonction principale qui appelle la fonction **f** avec une valeur saisie au clavier.

```
#include <iostream>
#include <variant>
std::variant<double, std::string> f(double& x) {
    double r = (x - 1) * (2 - x);

    if (r > 0)
        return sqrt(r);
    else
        return "Erreur sur le signe de l'expression !!";
}
```

UNE VARIABLE PLUSIEURS TYPE - EXERCICE

```
int main()
{
    double val;
    std::cout << "Donner une valeur:";
    std::cin >> val;
    auto resultat = f(val);
    if (resultat.index() == 0)
        std::cout << "f(" << val << ")=" << std::get<0>(resultat);
    else
        std::cout << std::get<1>(resultat);
    return 0;
}
```

Vérifier si le premier objet est actif

Accéder (lire) le contenu du variant

STD::TUPLE / STD::PAIR

En C++, `std::pair` et `std::tuple` servent à regrouper plusieurs valeurs dans une seule variable, mais avec des différences

- Un `std::pair` est utilisé pour combiner **exactement** deux objets qui peuvent être de types différents.

- `#include <utility>`

- On utilise `first` et `second` pour accéder aux objets contenus dans le `pair`

- Un `std::tuple` est un objet qui peut contenir plusieurs objets. Les objets peuvent être de différents types.

- Les objets contenus dans un `tuple` sont construits dans leur ordre d'accès.

- On accède aux éléments avec `std::get<index>(tuple)`

- `#include <tuple>`

```
#include <iostream>
#include <utility> // pour std::pair
using namespace std;

int main() {
    pair<int, string> p = {1, "Bonjour"};
    cout << p.first << " - " << p.second << endl;
}
```

```
#include <iostream>
#include <tuple>
using namespace std;

int main() {
    tuple<int, string, double> t = {42, "Salut", 3.14};

    cout << get<0>(t) << ", " << get<1>(t) << ", " << get<2>(t) << endl;
}
```

STRUCTURED BINDINGS / TUPLE / PAIR

```
#include <iostream>
#include <tuple>

std::tuple<std::string, std::string, int> creerMoi()
{
    return("Amani", "FALLAH", 2025)
}

int main()
{
    auto [prenom, nom, annee] = creerMoi();
    cout << prenom << " " << nom << " " << annee << endl;

    return 0;
}
```

Espace mémoire structuré
(en général
utilisé une seule fois, pour
éviter de
créer un objet `Personne`
par exemple)

STRUCTURED BINDINGS / TUPLE / PAIR

Exercice

Ecrire une fonction `décomposer` qui prend en paramètre une chaîne de caractère et retourne le jour, le mois et l'année séparément.

Ecrire une fonction principale `main` qui saisie la date sous forme `mm/jj/aaaa` et appelle la fonction `décomposer`, puis affiche le résultat.

On peut utiliser `std::stoi` pour convertir un `string` en un `int`

STRUCTURED BINDINGS / TUPLE / PAIR

Solution

```
#include <iostream>
#include <string>
#include <tuple>

std::tuple<int, int, int> decomposer(std::string& date) {
    int day = std::stoi(date.substr(0, 2));
    int month = std::stoi(date.substr(3, 5));
    int year = std::stoi(date.substr(6, 9));

    return { day, month, year };
}
```

```
int main()
{
    std::string maDate;
    std::cout << "Donner une date sous forme aa/jj/aaaa";
    std::cin >> maDate;
    auto [jour, mois, annee] = decomposer(maDate);
    std::cout << "Jour:" << jour << std::endl;
    std::cout << "Mois:" << mois << std::endl;
    std::cout << "Annee:" << annee << std::endl;
}
```

LVALUE et RVALUE

En général, on les utilise selon l'expression suivante:

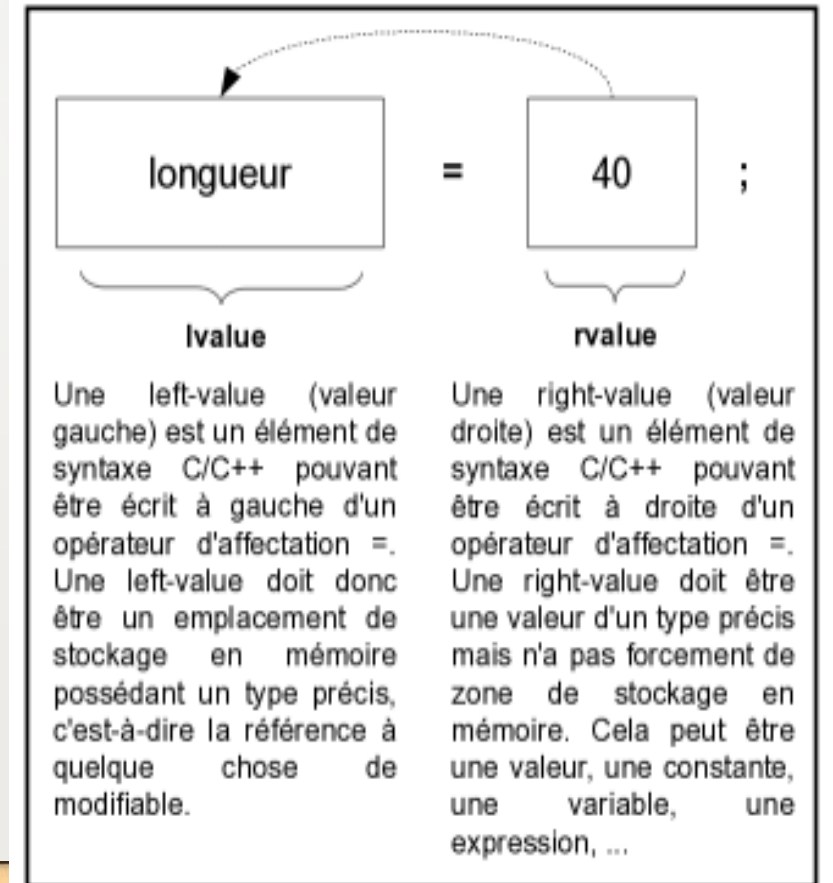
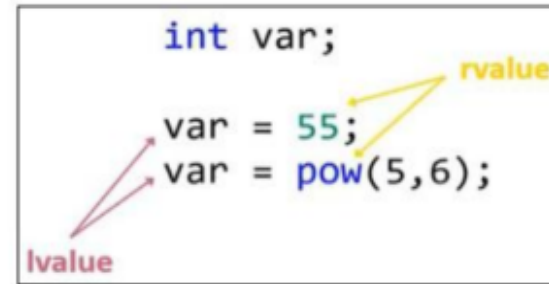
$\text{lvalue} = \text{rvalue}$

Lvalue est un objet qui a une location identifiée en mémoire (a une adresse en mémoire).

- Capable de stocker une information
- Ne peut pas être une fonction, une constante ou une expression

Rvalue est un objet n'ayant pas un identifiant en mémoire (désigne la valeur dans un espace mémoire).

- Toute chose pouvant retourner une expression constante ou une valeur.



LES REFERENCES EN C++

En C++, il est possible de déclarer une référence **j** sur une variable **i** : cela permet de créer **un nouveau nom j qui devient synonyme de i** (un alias). On pourra donc modifier le contenu de la variable en utilisant une référence. La déclaration d'une référence se fait en précisant le type de l'objet référencé, puis le symbole &, et le nom de la variable référence qu'on crée.

```
#include <iostream>

int main (int argc, char **argv)
{
    int i = 10; // i est un entier valant 10
    int &j = i; // j est une référence sur un entier, cet entier est i.
    //int &k = 44; // ligne7 : illégal

    std::cout << "i = " << i << std::endl; // i = 10
    std::cout << "j = " << j << std::endl; // j = 10

    // A partir d'ici j est synonyme de i, ainsi :
    j = j + 1; // est équivalent à i = i + 1 !

    std::cout << "i = " << i << std::endl; // i = 11
    std::cout << "j = " << j << std::endl; // j = 11

    return 0;
}
```

Remarque:

Une référence ne peut être initialisée qu'une seule fois : lors de sa déclaration. Une référence ne peut donc référencer qu'une seule variable tout au long de sa durée de vie.

LES REFERENCES EN C++

→ Intérêt des références :

- Comme une référence établit un lien entre deux noms, leur utilisation est efficace dans le cas de variable de grosse taille car cela évitera toute copie.
- Les références sont (systématiquement) utilisées dans le passage des paramètres d'une fonction (ou d'une méthode) dès que le coût d'une recopie par valeur est trop important ("gros" objet).

Remarque :

Une référence peut être déclarée constante ce qui empêchera toute modification par une fonction (ou une méthode).

TEMPLATES – FONCTIONS TEMPLATES

```
template<typename T>
T maxi(T a, T b){
    return a>b?a:b;
}

int main()
{
    float res3 = maxi(5.3,2.3);
    int res4 = maxi(10,12);
    cout << "float maxi: " << res3 << endl;
    cout << "int maxi: " << res4 << endl;
    return 0;
}
```

On peut définir plusieurs
parameters templates pour
une fonction

```
1 template <class T, class U>
2 T GetMin (T a, U b) {
3     return (a<b?a:b);
4 }
```

Les modèles de fonction sont des fonctions spéciales qui peuvent fonctionner avec des types génériques.

Un paramètre de modèle est un type spécial de paramètre qui peut être utilisé pour passer un type comme argument

On peut écrire aussi:

```
template<class T>
T maxi(T a, T b){
    return a>b?a:b;
}
```

POINTEURS SUR LES FONCTIONS

Un pointeur de fonction contient l'adresse du début du code binaire constituant la fonction.

```
#include <iostream>
void afficherInt(int a){
    std::cout << "c'est un " << a << std::endl;
}
int main()
{
    void(*aff)(int); // pointeur sur la fonction
    aff = afficherInt;
    aff(6);
    return 0;
}
```

Le nom de la fonction représente son adresse de début.

Nom de la fonction

type (*identificateur)(paramètres);

Les paramètre de la fonction

Type renvoyé par la fonction

POINTEURS SUR LES FONCTIONS - EXERCICE

Etant donné un tableau d'entiers `tab`, on se propose de faire différentes manipulations sur ce tableau:

- Ecrire une fonction « `affichePair` » qui affiche un entier s'il est pair.
- Ecrire une fonction « `afficheImpair` » qui affiche un entier s'il est impair.
- Ecrire une fonction « `pourChaque` » qui applique une fonction (passée en paramètre) sur les éléments des tableaux.
- Ecrire une fonction principale `main` qui crée un tableau d'entiers et affiche pour chaque élément pair du tableau puis chaque élément impair du tableau.

POINTEURS SUR LES FONCTIONS - CORRECTION

```
#include <iostream>
#include <vector>
void afficherPair(int a){
    if(!(a%2))
        std::cout << a << " est pair" << std::endl;
}
void afficherInPair(int a){
    if(a%2)
        std::cout << a << " est impair" << std::endl;
}
void PourChaque(const std::vector<int>& vec, void(*aff)(int)){
    for(int val : vec)
        aff(val);
}
int main()
{
    std::vector<int> tab = {3,9,2,5,7,4,12};
    PourChaque(tab, afficherPair);
    std::cin.get();
    PourChaque(tab, afficherInPair);
    return 0;
}
```


PASSAGE DE PARAMETRE – PASSAGE PAR VALEUR

Lorsque l'on passe une valeur en paramètre à une fonction, cette valeur est copiée dans une variable locale de la fonction correspondant à ce paramètre. Cela s'appelle un **passage par valeur**.

Exemple :

```
#include <iostream>
using namespace std;

void incrementer(int x) {
    x = x + 1;    // x est une COPIE
    cout << "Dans la fonction : x = " << x << endl;
}

int main() {
    int a = 5;
    incrementer(a); // a est passé PAR VALEUR
    cout << "Dans main : a = " << a << endl;
    return 0;
}
```

Résultat :

Dans la fonction : x = 6

Dans main : a = 5



Remarque : a reste 5 car la fonction a travaillé sur une copie.

PASSAGE DE PARAMETRE – PASSAGE PAR REFERENCE

La fonction reçoit **une référence** à la variable originale, **pas une copie**. Donc les changements dans la fonction affectent la variable d'origine.

```
void incrementer_ref(int &x) { // passage par référence
    x = x + 1;
}
```

```
int a = 5;
incrementer_ref(a);
cout << "a = " << a << endl; // a est maintenant 6
```

PASSAGE DE PARAMETRE – PASSAGE PAR POINTEUR

Similaire à référence, mais syntaxe différente

```
#include <iostream>
using namespace std;

// Fonction qui échange deux entiers via pointeurs
void echanger(int *x, int *y) {
    int temp = *x;    // on accède à la valeur pointée
    *x = *y;
    *y = temp;
}

int main() {
    int a = 10, b = 20;

    cout << "Avant échange : a = " << a << ", b = " << b << endl;

    // On passe l'adresse des variables
    echanger(&a, &b);

    cout << "Après échange : a = " << a << ", b = " << b << endl;

    return 0;
}
```

PASSAGE DE PARAMETRE – RESUME ET COMPARAISON

Méthode	Syntaxe de la fonction	Appel de la fonction	Effet sur la variable d'origine	Exemple
Par valeur	<code>void f(int x)</code>	<code>f(a);</code>	✅ Une copie est créée → l'original ne change pas	<pre>cpp void f(int x){ x++; } int a=5; f(a); // a=5</pre>
Par référence	<code>void f(int &x)</code>	<code>f(a);</code>	⚡ Pas de copie → l'original est modifié	<pre>cpp void f(int &x){ x++; } int a=5; f(a); // a=6</pre>
Par pointeur	<code>void f(int *x)</code>	<code>f(&a);</code>	⚡ L'adresse est transmise → l'original est modifié (il faut <code>*x</code> pour accéder)	<pre>cpp void f(int *x){ (*x)++; } int a=5; f(&a); // a=6</pre>